BOM-Track: Viva Notes (for me, not for submission)

What problem does the project solve?

A KiCad BOM is only a list. It cannot tell me the cost of a board, what to buy, or whether I can build it. BOM-Track stores the basic facts and lets SQL calculate those answers, so they are always current.

Where does the data come from?

My own uGameBox BOM (49 components) plus 10 published open-hardware boards from GitHub (Ubertooth family, OtterPill, OLIMEX ESP32-PoE-ISO), parsed from their KiCad schematics. Prices: 18 real Mouser prices from a 2010 BOM file and 5 INR prices I read from Probots and Zbotic. My extraction of the OLIMEX board matched their published BOM exactly (138 designators).

Why are build and stock_movement empty at the start?

Because I have not bought parts or built boards yet. Rows there would be made up. They are filled by using the app; those demo rows are labelled and can be removed with 99_reset_demo.sql.

Why is PLACEMENT a separate table?

The CSV Refs column holds a list ("R1-R8,R10"), which breaks 1NF. One row per designator fixes that. It also resolves the M:N between BOARD and PART and keeps the label (BTN0, RESET) for each position.

Why is there no quantity column, stock column or status column?

They can be calculated: quantity = COUNT of placements, stock = SUM of movements minus consumption, status = whether completed_on is NULL. Storing them would create two sources of truth that can drift apart.

What normal form are the tables in?

BCNF. In every table each determinant is a candidate key: the surrogate key, and natural keys like (board_id, ref_designator) and (category_id, value_text, footprint), which are declared UNIQUE.

Explain the cursor.

sp_shopping_list declares an explicit cursor over v_build_shortfall for one build, OPENs it, FETCHes row by row in a LOOP, looks up the latest quote for each part, prints a line with RAISE NOTICE, adds to the total, and returns total and unpriced count through INOUT parameters. Then CLOSE.

Explain the three triggers.

trg_stock_guard: BEFORE INSERT on stock_movement, stops stock going below zero. trg_build_complete: BEFORE UPDATE OF completed_on on build, rejects completion if any part is short. trg_freeze_bom: BEFORE INSERT/UPDATE/DELETE on placement, blocks

BOM edits after a board has been built, because past consumption is computed from the placements.

Why does trg_build_complete use SELECT ... FOR UPDATE?

Without it, two sessions completing two builds at once both see enough stock and both succeed: stock went to -9 in my demo (write skew). With the lock, the second session waits, then sees the first one's commit and is rejected.

What did the recovery demo show?

A committed purchase survived an immediate shutdown (durability, WAL redo on restart); an uncommitted one was gone (atomicity). The server log shows 'automatic recovery in progress' and 'redo starts at'.

Did the index make the query faster?

The plan changed from Seq Scan to Bitmap Index Scan and the cost estimate dropped from 12.47 to 9.02, but the timings are noise because the table is 678 rows in 4 pages. I did not fake a big table to show a speed-up.

Why PostgreSQL and not Oracle?

PL/pgSQL has the same features the guidelines ask for (procedures, functions, cursors, triggers, exceptions). It is free, runs on my laptop, and I could run and capture every demo.

How does the app talk to the database?

Python psycopg2. Every write runs in 'with conn:' which commits on success and rolls back on any exception. Errors raised by triggers are shown to the user, and each page shows the SQL it ran.

What would you improve?

Normalize values (100n = 100nF = 0.1uF), import LCSC part numbers and fetch prices, and reserve stock for planned builds.

Numbers to remember

Item	Value
Boards / parts / placements	11 / 242 / 678
Categories / suppliers / quotes	15 / 11 / 23
Queries / views	17 / 9
Functions / procedures / triggers	2 / 3 (1 with cursor) / 3
uGameBox priced cost	INR 315.84 for 4 of 20 lines
Concurrency without lock	stock -9 push buttons